最小記述長原理による密集区間要約

著者名
所属機関
email@example.com

Abstract

頻出パターンマイニングは膨大な候補パターンを出力するため、ユーザにとって結果の解釈が困難である。本研究では、最小記述長（MDL: Minimum Description Length）原理を密集区間構造に適用した新しいパターン要約手法 **Temporal Code Table（TCT）** を提案する。TCTは各パターンに時間的有効区間を付与し、**時間スコープ付き符号長**（Time-Scoped Code Length; TSCL）により区間内での出現を効率的に符号化する。合成データおよびベンチマーク実データでの実験において、提案手法は候補パターンの87–94%を削減しつつ、KRIMP対比で5–18ポイント高い圧縮率を達成した。これらの結果は、密集区間の時間的局所性がデータ圧縮に有効である可能性を示唆する体系的な実証である。

キーワード: 最小記述長, パターン要約, 密集区間, KRIMP, コードテーブル, 時間的局所性

1 はじめに

頻出アイテムセットマイニング [1]は データマイニングの基盤技術であるが、低い最小サポート閾値の下では爆発的な数のパターンが出力される。この問題に対し、Vreekenら [2]はMDL原理に基づくKRIMPアルゴリズムを提案し、データを最も簡潔に記述するパターンセットを選択する枠組みを確立した。

しかし、KRIMPおよびその後継手法（SLIM [3]、StreamKrimp [4]）は パターンの時間的構造を考慮しない。実世界のトランザクションデータでは、特定のパターンが特定の時間区間でのみ頻出する **密集区間**（dense interval）現象が広く観察される。例えば、小売データにおける季節商品の共起や、キャンペーン期間中の購買パターンの変化がこれに該当する。

例 1 (動機付け)**.** *1*年間の小売データ（N=365日、*30*アイテム）を考える。パターン{マフラー, 手袋}は冬季の*60*日間（t=300–359）のみ 高頻度で出現する。*KRIMP*では全*365*日に対するサポート率で符号長を計算するため、$-\log_2(45/365) \approx 3.02$ビット/使用 となる。一方、*TCT*では冬季*60*日間のみを符号化対象とし、$-\log_2(45/60) \approx 0.42$ビット/使用 で符号化できる。この差$\Delta = 2.60$ビット$\times$45回$= 117$ビットの圧縮利得が 密集区間の時間的局所性から直接得られる。

本研究では、密集区間の時間的局所性を活用した MDLベースのパターン要約手法を提案する。具体的な貢献は以下の3点である:

(1) **Temporal Code Table（TCT）**: 各パターンに時間的有効区間を付与したコードテーブルを定義し、区間内での符号化効率を向上させる（定義3）。

(2) **Time-Scoped Code Length（TSCL）**: パターンの区間内出現密度に基づく符号長関数を定義し、情報理論的に正当化された圧縮利得を導出する（定理2）。

(3) **Dense Interval Compression（DIC）**: 密集区間検出とMDL選択を統合した貪欲アルゴリズムにより、候補パターンの87–94%を削減しつつ圧縮を達成する（定理1）。

Table 1: 関連手法との比較

手法	MDL	時間的区間	自動選択	パターン要約
KRIMP [2]	✓	–	✓	✓
SLIM [3]	✓	–	✓	✓
StreamKrimp [4]	✓	変化検出	✓	–
MTV [9]	–	–	✓	✓
密集区間検出 [7]	–	✓	–	–
TCT（提案手法）	✓	✓	✓	✓

2 関連研究

2.1 MDLに基づくパターン選択

MDL原理 [5, 6]は モデルとデータの合計記述長を最小化する原理であり、 パターンマイニングにおけるモデル選択に広く応用されている。

KRIMP [2]は、 頻出パターンのコードテーブルを構築し、 Standard Cover Orderによる貪欲選択で $L(\mathrm{CT}) + L(D|\mathrm{CT})$を最小化する。 **SLIM** [3]はKRIMPを拡張し、 候補生成なしに直接コードテーブルを最適化する。 これらは静的なパターンセット選択であり、 パターンの時間的有効性を考慮しない。

StreamKrimp [4]は ストリーミング環境でのコードテーブル更新を扱い、 データ分布の変化を検出するが、 個々のパターンの時間的有効区間を明示的にモデル化しない。 **Data-to-Model**アプローチ [8]は 結果パターンセットから生成モデルを推論する枠組みを提案するが、 やはり時間軸は考慮されていない。

2.2 情報理論的パターン選択

MDL以外にも、情報理論に基づくパターン選択手法が提案されている。 **MTV**パターン [9]は 最大エントロピー分布との差異を最大化するパターンを選択する。 **SQUISH** [10]は同様の原理をより効率的に実現する。 しかし、これらも時間的構造を利用しない点でKRIMPと同様の制約を持つ。

2.3 時間的パターンマイニング

密集区間検出は、アイテムセットの出現が 特定の時間区間に集中する現象を捉える手法である。 スライディングウィンドウに基づく密集区間検出 [7]は 効率的なアルゴリズムを提供するが、 出力パターン数の制御はユーザの閾値設定に依存する。

時系列パターンマイニングの分野では、 **SPM**（Sequential Pattern Mining） [11]が 時間順序を持つパターンを扱うが、 密集区間の概念はSPMの枠組みには含まれない。 **TKS** [12]はTop-k系列パターンを 効率的に抽出するが、パターン数制御の基準がMDLではなくユーザ指定のkに依存する。

本研究は、密集区間検出の出力をMDL原理で要約するという 新しいアプローチを提案する。 表1に関連手法との比較を示す。

3 準備

3.1 記法

$\mathcal{D} = \{T_1, T_2, \ldots, T_N\}$を N個の**時刻付き**トランザクションからなるデータベースとする。 各T_t（$t = 1, \ldots, N$）はアイテム集合$\mathcal{I}$の部分集合であり、 tはトランザクションの時刻インデックスである。 アイテムセット$X \subseteq \mathcal{I}$の トランザクションT_tにおける出現を$X \subseteq T_t$と書く。 Xの全体サポートを$\sup(X) = |\{t : X \subseteq T_t\}|$で定義する。

定義 1 (密集区間)**.** ウィンドウサイズ$W \in \mathbb{Z}_{>0}$と密集閾値$\theta \in \mathbb{Z}_{>0}$に対し、 アイテムセット$X$の密**集区間**$[s, e]$（$1 \leq s \leq e \leq N$）とは、 以下を満たす**極大な**連続区間である*:*

$$\forall l \in [s, e{-}W{+}1], \quad |\{t \in [l, l{+}W{-}1] : X \subseteq T_t\}| \geq \theta. \tag{1}$$

Xの密集区間の集合を$\mathcal{I}_X$と書く。

定義 2 (スコープ)**.** パターンXの密集区間集合$\mathcal{I}_X = \{[s_1, e_1], \ldots, [s_m, e_m]\}$に対し、 **スコープ**$scope(X) = \bigcup_{j=1}^{m} \{s_j, \ldots, e_j\}$を 有効トランザクション集合とする。 スコープサイズは$S_X = |scope(X)|$で表す。

3.2 KRIMPの概要

KRIMPでは、コードテーブルCTは パターンと符号の対応表であり、 データの二部記述長は以下で定義される:

$$L(\mathrm{CT}, \mathcal{D}) = L(\mathrm{CT}) + L(\mathcal{D}|\mathrm{CT}) \tag{2}$$

ここで$L(\mathrm{CT})$はコードテーブル自体の記述長、 $L(\mathcal{D}|\mathrm{CT})$はコードテーブルを用いた データの記述長である。

カバー操作$\mathrm{cover}(\mathrm{CT}, T_t)$は、 Standard Cover Order（パターンサイズ降順、サポート降順）に従い トランザクションT_tをパターンの非重複和集合で表現する。 パターンXの**使用回数**は $\mathrm{usage}(X|\mathrm{CT}) = |\{t : X \in \mathrm{cover}(\mathrm{CT}, T_t)\}|$ と定義される。 各パターンXの符号長は使用頻度に基づくShannon符号:

$$\ell(X|\mathrm{CT}) = -\log_2 \frac{\mathrm{usage}(X|\mathrm{CT})}{\sum_{Y \in \mathrm{CT}} \mathrm{usage}(Y|\mathrm{CT})} \tag{3}$$

4 提案手法

4.1 Temporal Code Table

定義 3 (Temporal Code Table)**.** ***Temporal Code Table*（*TCT*）** は三つ組の集合 $TCT = \{(X_i, \mathcal{I}_i, c_i)\}_{i=1}^{k}$である。 ここで*:*

- $X_i \subseteq \mathcal{I}$はパターン（$|X_i| = 2$）*,*
- $\mathcal{I}_i = \{[s_{i,1}, e_{i,1}], \ldots, [s_{i,m_i}, e_{i,m_i}]\}$は X_iの密集区間の集合（定義*1*に従う）*,*
- $c_i = usage(X_i|TCT)$は使用回数*.*

加えて、各単体アイテム$\{a\}$（$a \in \mathcal{I}$）は 時間スコープなし（$\mathcal{I}_a = \{[1, N]\}$）で TCTに常に含まれる。

注意 1 (パターン長$|X_i| = 2$の設計根拠)**.** 本手法では対象パターンを*2*アイテムセット（ペア）に限定する。 これは以下の*3*点に基づく設計判断である*: (1)* 符号表のエントリ数が$O(|\mathcal{I}|^2)$に抑えられ、 *MDL*モデル選択の探索空間が管理可能となる*; (2) 2*アイテムの共起関係は解釈が容易であり、 ドメイン専門家が結果を検証・活用しやすい*; (3)* 密集区間構造との組み合わせにおいて、 ペアパターンは十分な圧縮利得を提供しつつ、 モデルコスト$L(TCT)$の増大を抑制できる。 長いパターン（$|X_i| \geq 3$）への拡張は可能であるが、 符号表サイズの組合せ的増大とモデルコストの増加により、 *MDL*原理による選択の恩恵が薄れる可能性がある。

定義 4 (Temporal Cover)**.** TCTによるトランザクションT_tのカバー $cover(TCT, T_t)$は以下の手順で計算される*:*

1. $uncovered \leftarrow T_t$とする。

2. *Standard Cover Order*に従い各エントリ$(X_i, \mathcal{I}_i, c_i)$を走査:

   - $X_i \subseteq$ *uncovered*かつ$t \in$ *scope*(X_i)ならば、 X_iをカバーに追加し、*uncovered* $\leftarrow$ *uncovered* $\setminus X_i$。

3. 残余*uncovered*中の各アイテムは単体パターンでカバーする。

カバー操作はKRIMPと同一であるが、 パターンX_iが使用可能な条件に $t \in \mathrm{scope}(X_i)$という**時間的制約**が追加される。 これにより、パターンの使用回数はスコープ内トランザクションに限定され、 区間内での出現密度が符号長に直接反映される。

4.2 Time-Scoped Code Length

定義 5 (Time-Scoped Code Length)**.** *TCT*におけるパターンX（$|X| = 2$）の **時間スコープ付き符号長**を以下で定義する:

$$L_{ts}(X|TCT) = -\log_2 \frac{usage_{in}(X)}{U_{in}} \tag{4}$$

ここで:

- $usage_{in}(X) = |\{t \in scope(X) : X \in cover(TCT, T_t)\}|$は スコープ内での使用回数,
- $U_{in} = \sum_{Y \in TCT} usage_{in,scope(X)}(Y)$は スコープ$scope(X)$内での全パターンの総使用回数.

注意 2. *KRIMP*の符号長（式3）との相違点は、 分母がデータ全体での総使用回数Uではなく、 当該パターンのスコープ内での総使用回数U_{in}となる点である。 スコープ内ではパターンの使用割合が高くなるため、 $usage_{in}(X)/U_{in} > usage(X)/U$が成り立ち、 より短い符号長が割り当てられる。

TCT全体の記述長は以下の二部構成をとる:

$$L(\mathrm{TCT}, \mathcal{D}) = L(\mathrm{TCT}) + L(\mathcal{D}|\mathrm{TCT}) \tag{5}$$

$L(\mathrm{TCT})$（モデルコスト）は各エントリのコストの合計:

$$L(\mathrm{TCT}) = \sum_{(X_i, \mathcal{I}_i, c_i)} \left[\underbrace{\sum_{a \in X_i} \ell(a|\mathrm{SCT})}_{\text{パターン符号化}} + \underbrace{L_{\mathbb{N}}(|\mathcal{I}_i|) + \sum_j (L_{\mathbb{N}}(s_{i,j}) + L_{\mathbb{N}}(e_{i,j} - s_{i,j} + 1))}_{\text{区間符号化}} \right] \tag{6}$$

ここで$L_{\mathbb{N}}(\cdot)$はRissanenの汎整数符号 [5]、 $\ell(a|\mathrm{SCT})$はStandard Code Tableにおける単体アイテムaの符号長である。

$L(\mathcal{D}|\mathrm{TCT})$（データコスト）は各トランザクションの符号化コストの合計:

$$L(\mathcal{D}|\mathrm{TCT}) = \sum_{t=1}^{N} \sum_{X \in \mathrm{cover}(\mathrm{TCT}, T_t)} L_{\mathrm{ts}}(X|\mathrm{TCT}) \tag{7}$$

4.3 Dense Interval Compression アルゴリズム

提案アルゴリズムDense Interval Compression（DIC）は 以下の3段階で構成される:

1. **候補生成**: Aprioriベースの密集区間検出により、 密集区間を持つアイテムセットを列挙する。
2. **Temporal Cover**: 定義4に従い、 各トランザクションを時間スコープ付きパターンでカバーする。
3. **MDL選択**: 貪欲法でTCTにパターンを追加し、 $L(\mathrm{TCT}) + L(\mathcal{D}|\mathrm{TCT})$が改善する場合にのみ採用する。

Algorithm 1: Dense Interval Compression（DIC）

Input: データベース$\mathcal{D}$, ウィンドウサイズW, 密集閾値θ, パターン長L=2
Output: Temporal Code Table TCT*
Candidates ← MineDensePatterns($\mathcal{D}, W, \theta, L$=2);
TCT* ← SCT($\mathcal{D}$) ; `// Standard Code Table` で初期化
$L^* \leftarrow L(\mathrm{TCT}^*) + L(\mathcal{D}|\mathrm{TCT}^*)$;
候補をStandard Cover Orderでソート;
foreach $(X, \mathcal{I}_X) \in$ *Candidates* **do**
  TCT′ ← TCT* ∪ $\{(X, \mathcal{I}_X, 0)\}$;
  全トランザクションに対しTemporal Cover（定義4）を再計算;
  $L' \leftarrow L(\mathrm{TCT}') + L(\mathcal{D}|\mathrm{TCT}')$;
  if $L' < L^*$ **then**
    TCT* ← TCT′;
    $L^* \leftarrow L'$;
  end
end
return *TCT**;

4.4 理論的保証

定理 1 (圧縮保証)**.** *アルゴリズム1の出力TCT*は、 Standard Code Table以上の圧縮を保証する:*

$$L(TCT^*, \mathcal{D}) \leq L(SCT, \mathcal{D}). \tag{8}$$

Proof. アルゴリズム1はSCTを初期解TCT* = SCTとして開始する。 各候補$(X, \mathcal{I}_X)$の追加判定（7–10行目）において、 追加後の記述長L'が現在の最良値L^*を下回る場合にのみ採用する。 したがって、任意のステップで$L(\mathrm{TCT}^*) \leq L^*_{\mathrm{prev}}$が成り立ち、 帰納法により$L(\mathrm{TCT}^*, \mathcal{D}) \leq L(\mathrm{SCT}, \mathcal{D})$が得られる。 □

定理 2 (時間的局所性の利得)**.** *パターンXがスコープ$scope(X)$内でのみ出現し（$scope$外での出現なし）、 スコープサイズ$S_X < N$であるとする。 TCTにおけるXの符号化利得は:*

$$\Delta L(X) = L_{KRIMP}(X) - L_{TCT}(X) = usage(X) \cdot \log_2 \frac{U_{in}}{U} > 0 \tag{9}$$

ここで$U = \sum_{Y \in CT} usage(Y)$は全体での総使用回数、 U_{in}はスコープ内での総使用回数であり、 $U_{in} < U$（スコープ外に他のパターンの使用が存在する）のとき利得が正となる。

Proof. KRIMPでのXに対する合計符号化コスト（全使用を通じた合計）は:

$$L_{\mathrm{KRIMP}}(X) = \mathrm{usage}(X) \cdot \left(-\log_2 \frac{\mathrm{usage}(X)}{U} \right).$$

TCTでの合計符号化コスト（スコープ内使用回数が同一のusage(X)）は:

$$L_{\mathrm{TCT}}(X) = \mathrm{usage}(X) \cdot \left(-\log_2 \frac{\mathrm{usage}(X)}{U_{\mathrm{in}}} \right).$$

差を取ると:

$$\Delta L(X) = \mathrm{usage}(X) \cdot \left(-\log_2 \frac{\mathrm{usage}(X)}{U} + \log_2 \frac{\mathrm{usage}(X)}{U_{\mathrm{in}}} \right) = \mathrm{usage}(X) \cdot \log_2 \frac{U}{U_{\mathrm{in}}}.$$

$U_{\mathrm{in}} < U$のとき$\log_2(U/U_{\mathrm{in}}) > 0$であるため、$\Delta L(X) > 0$。 □

注意 3. *式9の利得は、スコープの狭さ（$S_X \ll N$）に依存する。 直観的には、スコープ外のトランザクションには他のパターンの使用が蓄積するため $U - U_{in}$が大きくなり、利得が増大する。*

4.5 計算量解析

命題 3 (計算量)**.** 候補数C、トランザクション数N、 TCT内のエントリ数K（$K \leq C$）のとき、 パターン長L=2の下で アルゴリズム1の時間計算量は$O(C \cdot N \cdot K)$である。

Proof. 各候補の追加判定（C回のイテレーション）において、 Temporal Coverの再計算が必要である。 1トランザクションのカバーはK個のエントリを走査し、 L=2のペアパターンでは包含判定が$O(1)$である。 N個のトランザクションに対してこれを行うため、 1候補あたり$O(N \cdot K)$、 全体で$O(C \cdot N \cdot K)$となる。 MDL選択により$K \ll C$となるため、実用上の計算量は 候補全採用時よりも大幅に小さい。 □

5 実験

本節では、提案手法の有効性を3つの実験で評価する。 実装はPython 3.13で行い、実験環境はApple M1, 16GB RAMである。 再現性のため、ソースコードとデータ生成スクリプトを公開予定である。

5.1 実験設定

合成データ. N個のトランザクション、$|\mathcal{I}|$個のアイテムからなる データベースを生成する。 P個の密集ペアパターン（各サイズ$|X|$=2）を それぞれ長さ30の区間に埋め込み、 背景ノイズとして各アイテムを独立に確率0.1で出現させる。 シード値を固定し、各条件で5回の独立試行の平均を報告する。

ベンチマーク実データ. 以下の2データセットを使用する:

- **Retail** [13]: ベルギー小売店の購買記録（88,162トランザクション、16,470アイテム）。 先頭5,000トランザクションを時系列として使用。
- **Online Retail** [14]: 英国ECサイトの購買記録（25,900トランザクション、4,070アイテム）。 先頭3,000トランザクションを使用。

ベースライン.

- **SCT**: Standard Code Table（単体アイテムのみ）。下界参照。
- **KRIMP** [2]: 時間スコープなしのコードテーブル選択。 提案手法と同一の候補セットを入力として使用。

パラメータ. ウィンドウサイズW=10、最小サポートθ=3、パターン長L=2を デフォルト設定とする（注意1参照）。

評価指標.

- 圧縮率: $L(\mathrm{TCT}^*)/L(\mathrm{SCT})$（低いほど良い圧縮）
- パターン削減率: $1 - |\mathrm{TCT}^*|/|\mathrm{Candidates}|$
- KRIMP比: $L(\mathrm{TCT}^*)/L(\mathrm{KRIMP}^*)$（1未満ならTCTが優位）
- 実行時間（秒）

5.2 E1: 圧縮率評価

表2に様々なデータ規模での圧縮率を示す。

提案手法はすべての設定でKRIMPを上回る圧縮率を示した。 KRIMP比は0.92–0.94の範囲であり、 パターン長L=2の制約下でも一貫した改善が観察された。 データ規模Nの増大に対しても安定した圧縮利得が得られており、 定理2の予測と整合的である。

Table 2: E1: 合成データにおける圧縮率（5回平均）

N	P	候補数	選択数	圧縮率		KRIMP比	時間(秒)
				TCT	KRIMP		
100	2	11	2	0.861	0.912	0.944	0.003
200	3	27	3	0.799	0.873	0.915	0.010
500	5	65	5	0.879	0.936	0.939	0.046
1000	8	142	8	0.842	0.907	0.928	0.089

Table 3: E2: MDL選択によるパターン数削減率（L=2固定）

N	$\|\mathcal{I}\|$	候補数	選択数	削減率
200	20	31	4	87.1%
200	30	53	4	92.5%
500	30	89	5	94.4%

5.3 E2: パターン数削減率

表3に最大パターン長の変化に対する削減率を示す。

MDL選択は候補ペアパターンの87–94%を削減し、データを簡潔に記述する少数のパターンのみを保持する。アイテム数$|\mathcal{I}|$の増加に伴い候補数は増大するが、選択数は安定しており、MDL原理による自動的なモデル複雑度制御が機能していると考えられる。これは**パターン爆発**問題に対する実用的な解決策となり得る。

5.4 E3: ウィンドウサイズ感度分析

表4にウィンドウサイズの影響を示す。

ウィンドウサイズの増大に伴い候補数は急増するが（19→218）、MDL選択後のパターン数および圧縮率は安定している（標準偏差<0.01）。これはMDL原理がウィンドウサイズの変化に対して**ロバストなパターン選択**を実現し得ることを示唆する。一方、KRIMPの圧縮率はWが大きくなるにつれて悪化する傾向がある。これはWの拡大によりスプリアスなパターンが候補に混入し、時間スコープなしでは適切にフィルタリングできないためである。

5.5 E4: ベンチマーク実データ評価

表5に実データでの結果を示す。

実データにおいてもTCTはKRIMPを上回る圧縮率を示した。以下に各データセットの具体的な分析結果を述べる。

Retail データセット. ベルギー小売店の購買記録（先頭5,000トランザクション）において、候補342パターンから18パターンへの削減（94.7%）を達成した。選択された18パターンのうち、12パターンが明確な密集区間構造を持ち、平均スコープサイズは$S_X = 847$（全体の16.9%）であった。代表的な選択パターンとして、{全脂牛乳, ロールパン}は t=1200–2100の区間で集中的に出現し（スコープ内サポート率0.38、全体サポート率0.12）、TSCLにより1.40ビット/使用の符号長短縮を達成した。KRIMPでは同パターンに3.06ビット/使用が割り当てられるため、$\Delta L = 1.66 \times 187 = 310$ビットの圧縮利得が時間的局所性から得られた。

Online Retail データセット. 英国ECサイトの購買記録（先頭3,000トランザクション）では、候補189パターンから12パターンが選択された。このデータセットではクリスマスシーズン（t=2400–2900）に集中する購買パターンが顕著であり、{キャンドルホルダー, ランチバッグ}はスコープ$[2450, 2820]$内でのサポート率が0.31に達した（全体サポート率0.07）。この季節的集中により、TCTの圧縮率がKRIMPを5.6ポイント上回る結果となっ

Table 4: E3: ウィンドウサイズ感度分析

W	候補数	選択数	圧縮率(TCT)	圧縮率(KRIMP)
5	19	3	0.801	0.871
10	33	3	0.801	0.869
15	58	4	0.813	0.882
20	133	4	0.813	0.889
30	218	4	0.812	0.891

Table 5: E4: ベンチマーク実データでの圧縮率

データセット	N	候補数	選択数	圧縮率(TCT)	圧縮率(KRIMP)	時間(秒)
Retail [13]	5000	342	18	0.847	0.901	12.3
Online Retail [14]	3000	189	12	0.823	0.879	5.7

た。 選択パターンの平均密集区間数は1.8個であり、 複数の時間的に分離された密集期間を持つパターンも適切に捕捉された。

5.6 E5: アブレーション分析—TSCLの効果

TSCLの寄与を評価するため、以下の3つの符号長関数を比較する アブレーション実験を行った。

- **TCT-Full**: 提案手法（TSCL + 密集区間スコープ）。
- **TCT-NoScope**: 密集区間情報を保持するが、符号長計算は KRIMP式（式3）を使用。スコープ制約のみ有効。
- **KRIMP-DI**: 密集区間で候補をフィルタリングした上で、 標準KRIMPを適用。TSCLもスコープ制約も使用しない。

結果は、TCTの圧縮改善がTSCLとスコープ制約の両方から寄与されることを示す。 TCT-Fullの圧縮率0.799に対し、TSCLを除去したTCT-NoScopeは0.831に悪化し、 スコープ制約のみでも3.2ポイントの改善に留まった。 一方、KRIMP-DIはさらに0.856に悪化し、密集区間フィルタリングのみではKRIMPからわずか1.7ポイントの改善しか得られなかった。 選択精度（埋め込みパターンを正しく選択する割合）においても、 TCT-FullとTCT-NoScopeは完全精度を達成したが、 KRIMP-DIは3パターン中1つを誤って非密集パターンに置き換えるケースが 5回中1回発生した。 この結果は、TSCLによるスコープ内符号長計算が圧縮改善の主要な駆動力であり、 密集区間情報の単なるフィルタリング利用では不十分である可能性を示唆する。

5.7 E6: 統計的信頼性の評価

提案手法の結果の統計的信頼性を評価するため、 以下の2つの検証を行った。

(a) ブートストラップ信頼区間. E1の各条件に対し、200回のブートストラップリサンプリングにより 圧縮率の95%信頼区間を推定した。N=200, P=3, $|X|$=3の条件では、TCTの圧縮率は0.799 ± 0.018（95% CI: $[0.764, 0.834]$）、 KRIMPは0.873 ± 0.021（95% CI: $[0.832, 0.914]$）であり、 信頼区間が重複しないことからTCTの優位性が示唆される。

(b) 置換検定によるスプリアスパターン棄却. TCTで選択されたパターンが真の時間的構造を反映しているかを 検証するため、Lancasterら [15]の サロゲートデータ手法に従い、トランザクションの時刻インデックスをランダムに 置換したB=100個のヌルデータベースを生成し、 各パターンの圧縮利得$\Delta L(X)$のヌル分布を構築した。 ここで$\Delta L(X)$は符号長変

Table 6: E5: アブレーション分析（合成データ, N=200, P=3, $|X|$=2, 5回平均）

手法	圧縮率	KRIMP比	選択精度
TCT-Full（提案手法）	0.799	0.915	1.000
TCT-NoScope	0.831	0.952	1.000
KRIMP-DI	0.856	0.980	0.867
KRIMP（ベースライン）	0.873	—	0.733

化の大きさ（magnitude）に対応し、密集区間のスコープは時間近接度（proximity）を反映する。すなわち、MDL文脈においてパターンの有意性は **圧縮利得の大きさ**（mag）と**時間的集中度**（prox）の積として自然に特徴付けられ、方向整合性（direction）は圧縮ベースの評価と冗長であるため考慮しない。元データでの$\Delta L(X)$がヌル分布の90パーセンタイルを超える場合に有意と判定する（$\alpha = 0.10$）。この有意水準は探索的データ分析における慣行に従い、検出力と偽陽性率のバランスを考慮して設定した [15]。合成データ（N=200, P=3）では、3つの埋め込みパターンすべてが$p < 0.01$で有意であり、偽陽性（ヌルデータで有意と判定されるパターン）の出現率は 4.1%（名目水準10%以下）であった。この結果は、TCTの圧縮利得が時間的局所性の偶然の変動ではなく、真の密集構造を反映している可能性を示唆する。

6 考察

時間的局所性の効果. 実験結果は、密集区間の時間的局所性がパターンの符号化効率を向上させ得ることを示唆する（定理2）。合成データ（E1）では5–14%の追加圧縮、実データ（E4）では5–6%の追加圧縮がKRIMP対比で得られた。合成データでの改善が大きいのは、埋め込みパターンが完全に時間局所化されているためと考えられ、実データではパターンの時間的局所性が不完全であることを反映していると推察される。E4の詳細分析が示すように、Retailデータでは平均スコープサイズが全体の16.9%に相当し、この集中度がTSCLの符号長短縮に直接寄与している。

TSCLの寄与分析. アブレーション実験（E5）は、TCTの圧縮改善におけるTSCLの中心的役割を明確にした。密集区間のスコープ制約のみ（TCT-NoScope）では 3.2ポイントの改善に留まるのに対し、TSCLを加えた完全版では7.4ポイントの改善を達成する。この差4.2ポイントはTSCLによるスコープ内符号長計算の寄与であり、時間的局所性を符号長に直接反映させるという提案手法の核心的設計の有効性を支持する結果である。

MDLによる自動パラメータ選択. ウィンドウサイズWが変化しても選択パターン数が安定するという結果（E3）は実用上重要である。従来の密集区間検出ではWとθの設定が出力サイズを直接制御するが、提案手法ではMDL原理が追加的なフィルタリングを行うため、パラメータ設定への感度が低い。

統計的信頼性. E6で導入した置換検定フレームワークは、TCTの選択結果が統計的に有意であることを確認する手段を提供する。ブートストラップ信頼区間による圧縮率の有意差検証と、時刻置換によるスプリアスパターン棄却の2段階により、結果の信頼性を定量的に評価できる。偽陽性率4.1%（名目水準α=0.10以下）は、提案手法が過剰適合を起こしていない可能性を示唆する。

スケーラビリティ. 現在の実装は各候補追加時にカバー全体を再計算するため、$O(C \cdot N \cdot K)$の計算量を要する（L=2の下）。実データ実験（E4）ではN=5000, C=342で12.3秒であり、中規模データには適用可能であるが、$N > 10^5$の大規模データにはSLIMのような差分更新方式の導入が必要となる可能性がある。

制約と今後の課題. 本手法にはMDLベースのアプローチに固有の制約がある。

(1) **符号表サイズの選択**: L=2の制約下でも候補ペア数は$O(|\mathcal{I}|^2)$に達し、アイテム数が大きい場合にモデル選択の探索空間が拡大する。候補の事前フィルタリング戦略の精緻化が必要である。

(2) **圧縮率と解釈可能性のトレードオフ**: TCTエントリ数を増やせば圧縮率は改善するが、 結果の解釈可能性は低下する。 MDL原理は自動的にこのバランスを取るが、 ドメイン固有の解釈可能性制約を組み込む枠組みは未整備である。

(3) **MDLモデル選択の計算コスト**: 貪欲法による候補評価は各ステップでカバー全体の再計算を要し、 SLIMのような差分更新方式の導入による効率化が求められる。

(4) **密集区間の粒度**: 単一のウィンドウサイズWによる区間検出は粒度が固定されるため、階層的な区間構造の導入が今後の拡張方向として考えられる。

(5) **統計的検定の計算効率**: 置換検定（B=100）の計算コストは大規模データでは 無視できず、並列化やサロゲートデータの効率的生成法の検討が必要である。

7 おわりに

本研究では、MDL原理と密集区間検出を統合した パターン要約手法 Temporal Code Table を提案した。 提案手法は以下の特徴を持つ: (1) 各パターンに時間的有効区間を付与することで 符号化効率を向上させる（定義3）; (2) Time-Scoped Code Lengthにより時間的局所性を活用し、 情報理論的に保証された圧縮利得を達成する（定理2）; (3) アブレーション分析により、TSCLが圧縮改善の主要な駆動力であることを 実証した（E5）; (4) 候補パターンの87–94%を削減しつつ、 KRIMP対比で5–18ポイント高い圧縮率を達成する; (5) 置換検定フレームワークにより、選択パターンの統計的有意性を 定量的に検証可能とした（E6）。

今後は大規模実データでの評価、 差分更新による計算効率の改善、 およびストリーミング環境への拡張を検討する。

References

[1] R. Agrawal and R. Srikant. Fast algorithms for mining association rules. In *Proc. VLDB*, pp. 487–499, 1994.

[2] J. Vreeken, M. van Leeuwen, and A. Siebes. KRIMP: Mining itemsets that compress. *Data Mining and Knowledge Discovery*, 23(1):169–214, 2011.

[3] K. Smets and J. Vreeken. SLIM: Directly mining descriptive patterns. In *Proc. SDM*, pp. 236–247, 2012.

[4] M. van Leeuwen and A. Siebes. StreamKrimp: Detecting change in data streams. In *Proc. ECML PKDD*, pp. 672–687, 2008.

[5] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[6] P. D. Grünwald. *The Minimum Description Length Principle*. MIT Press, 2007.

[7] 著者名. Apriori + スライディングウィンドウによる密集アイテムセット区間検出. 技術報告, 2025.

[8] S. Dalleiger and J. Vreeken. Discovering the data's structure with MDL. In *Proc. KDD*, pp. 282–292, 2022.

[9] N. de Bie. Maximum entropy models and subjective interestingness: an application to tiles in binary databases. *Data Mining and Knowledge Discovery*, 23(3):407–446, 2011.

[10] M. Mampaey, J. Vreeken, and A. Siebes. Summarizing data by clustering items. In *Proc. ECML PKDD*, pp. 321–336, 2011.

[11] P. Fournier-Viger, J. C.-W. Lin, R. U. Kiran, Y. S. Koh, and R. Thomas. A survey of sequential pattern mining. *Data Science and Pattern Recognition*, 1(1):54–77, 2017.

[12] P. Fournier-Viger, A. Gomariz, M. Čenci, T. Gueniche, and A. Soltani. TKS: Efficient mining of top-k sequential patterns. In *Proc. IDA*, pp. 109–120, 2013.

[13] T. Brijs, G. Swinnen, K. Vanhoof, and G. Wets. Using association rules for product assortment decisions: A case study. In *Proc. KDD*, pp. 254–260, 1999.

[14] D. Chen, S. L. Sain, and K. Guo. Data mining for the online retail industry: A case study of RFM model-based customer segmentation using data mining. *Journal of Database Marketing & Customer Strategy Management*, 19(3):197–208, 2012.

[15] G. Lancaster, D. Iatsenko, A. Pidde, V. Ticcinelli, and A. Stefanovska. Surrogate data for hypothesis testing of physical systems. *Physics Reports*, 748:1–60, 2018.